

National Tsing Hua University
Department of Electrical Engineering
EE429200 IC Design Laboratory, Fall 2025

**Lab 11: APR Flow with Innovus –
Design Setup and Floorplan (1%)**

Assigned on Nov. 13, 2025

Due day in **Nov. 20, 2025**

Objective

In this lab, you will learn how to use Innovus to perform a basic APR flow.

Demo checklist:

- ☐ Layout after floorplan
- ☐ Congestion map
- ☐ Questions:
 1. What is MMMC?
 2. What is core utilization?
 3. Why do we need to rearrange the IO orders in the .io file?

Environment Setup

1. Copy lab file packages from `~ee4292/`. Decompress the package and enter it.

```
$ cp ~ee4292/iclab2025/lab11.zip . (Don't forget the dot ".")
```

```
$ unzip lab11.zip
```

```
$ cd lab11/innovus/run
```

Data Preparation (already provided)

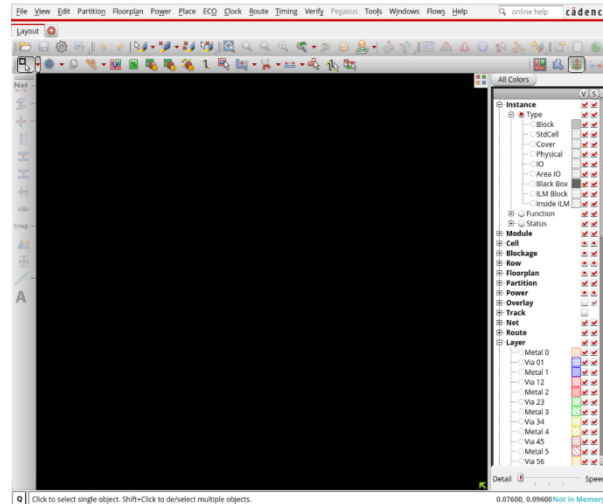
Files	Location	Description
Gate Level Netlist	<code>../pre_layout/top_pipe_syn.v</code>	Design netlist
Timing Constraint File	<code>../pre_layout/top_pipe_syn.sdc</code>	Define APR constraints
MMMC File	<code>../pre_layout/mmmc.view</code>	Define timing corners
QX Tech File	<code>/usr/cadtool/GPDK45/gsclib045_svt_v4.4/gsclib045/qrc/qx/gpdk045.tch</code>	Library for extract RC information
Timing Libraries	<code>/usr/cadtool/GPDK45/gsclib045_svt_v4.4/gsclib045/lib/*.lib</code>	Timing library for different corners
SI libraries	<code>/usr/cadtool/GPDK45/gsclib045_svt_v4.4/gsclib045/celtic/*.cdb</code>	Signal integrity libraries

Action Items

I. Design Setup.

1. In the **run/**, open the GUI of Innovus by:

```
$ innovus
```



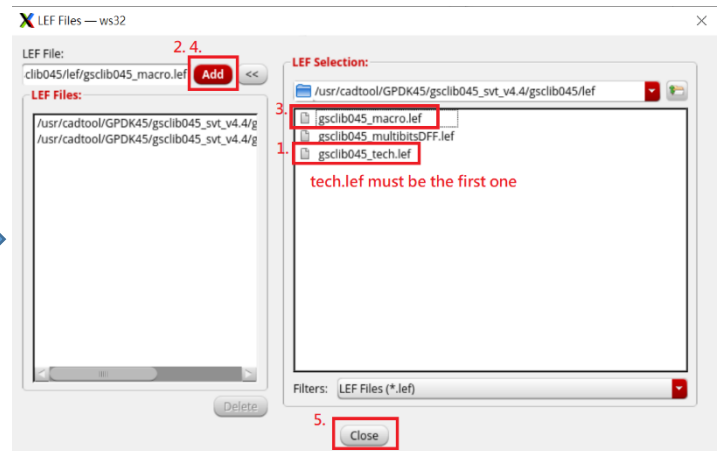
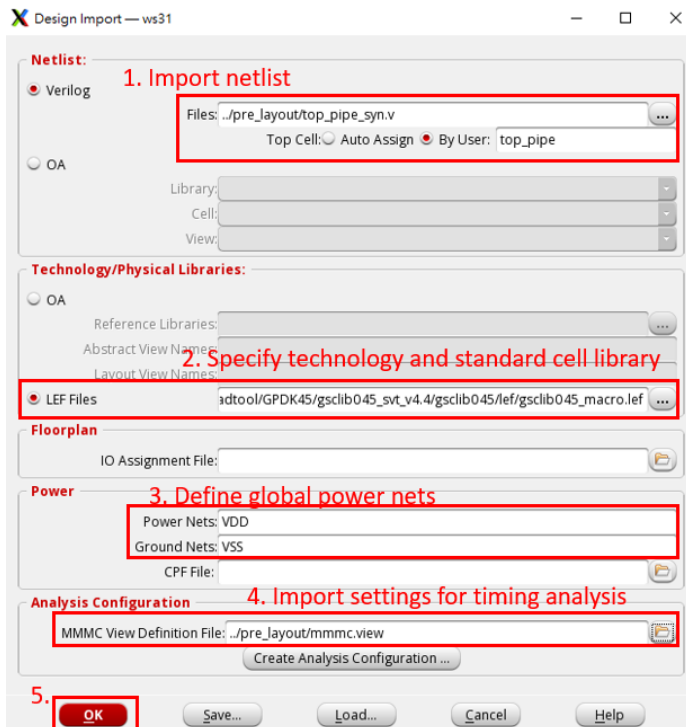
Note: (Optional) If you can't open INNOVUS, check .tcshrc file in your home directory and add the following command:

```
$ source /usr/cadtool/user_setup/03-innovus.csh
```

2. Import Design

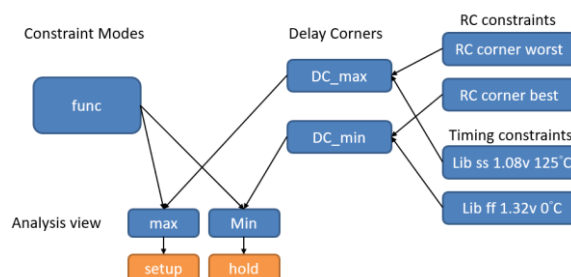
“File->Import Design...”

Verilog	../pre_layout/top_pipe_syn.v
LEF files	/usr/cadtool/GPDK45/gsclib045_svt_v4.4/gsclib045/lef/ gsclib045_tech.lef /usr/cadtool/GPDK45/gsclib045_svt_v4.4/gsclib045/lef/ gsclib045_macro.lef
Power	Power Nets: VDD Ground Nets: VSS
MMMC	../pre_layout/mmmc.view



Understand the settings of Multi-Mode Multi-Corner (mmmc)

I. Read `../pre_layout/mmmc.view` and understand the analysis view

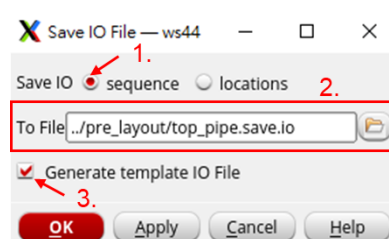


II. Read `../pre_layout/top_pipe_syn.sdc`, compare with the one in lab11/syn/netlist and see what has been modified

3. Create IO constraint file

“File->Save->I/O File...”

This step generates an IO template according to the IO declaration orders in your netlist.



4. Modify IO constraint file

Clock and reset signal should be put in the middle of a side for easier balancing.

```

14 (top
15 (pin name="clk" layer=2 width=0.0800 depth=0.2500 place_status=placed )
16 (pin name="rst_n" layer=2 width=0.0800 depth=0.2500 place_status=placed )
17 (pin name="boot_up" layer=2 width=0.0800 depth=0.2500 place_status=placed )
18 (pin name="boot_addr[7]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
19 (pin name="boot_addr[6]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
20 (pin name="boot_addr[5]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
21 (pin name="boot_addr[4]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
22 (pin name="boot_addr[3]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
23 (pin name="boot_addr[2]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
24 (pin name="boot_addr[1]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
25 (pin name="boot_addr[0]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
26 (pin name="boot_datai[31]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
27 (pin name="boot_datai[30]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
28 (pin name="boot_datai[29]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
29 (pin name="boot_datai[28]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
30 (pin name="boot_datai[27]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
31 (pin name="boot_datai[26]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
32 (pin name="boot_datai[25]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
33 (pin name="boot_datai[24]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
34 (pin name="boot_datai[23]" layer=2 width=0.0800 depth=0.2500 place_status=placed )
35 )
36 (left
37 (pin name="boot_datai[22]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
38 (pin name="boot_datai[21]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
39 (pin name="boot_datai[20]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
40 (pin name="boot_datai[19]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
41 (pin name="boot_datai[18]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
42 (pin name="boot_datai[17]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
43 (pin name="boot_datai[16]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
44 (pin name="boot_datai[15]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
45 (pin name="boot_datai[14]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
46 (pin name="boot_datai[13]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
47 (pin name="boot_datai[12]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
48 (pin name="boot_datai[11]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
49 (pin name="boot_datai[10]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
50 (pin name="boot_datai[9]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
51 (pin name="boot_datai[8]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
52 (pin name="boot_datai[7]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
53 (pin name="boot_datai[6]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
54 (pin name="boot_datai[5]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
55 (pin name="boot_datai[4]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
56 (pin name="boot_datai[3]" layer=3 width=0.0800 depth=0.2500 place_status=placed )
57 )
)

13 (icopin
14 (top
15 (pin name="boot_up" layer=2 width=0.0800 depth=0.2500 )
16 (pin name="boot_addr[7]" layer=2 width=0.0800 depth=0.2500 )
17 (pin name="boot_addr[6]" layer=2 width=0.0800 depth=0.2500 )
18 (pin name="boot_addr[5]" layer=2 width=0.0800 )
19 (pin name="boot_addr[4]" layer=2 width=0.0800 )
20 (pin name="boot_addr[3]" layer=2 width=0.0800 )
21 (pin name="boot_addr[2]" layer=2 width=0.0800 )
22 (pin name="boot_addr[1]" layer=2 width=0.0800 )
23 (pin name="boot_addr[0]" layer=2 width=0.0800 depth=0.2500 )
24 (pin name="boot_datai[31]" layer=2 width=0.0800 depth=0.2500 )
25 (pin name="boot_datai[30]" layer=2 width=0.0800 depth=0.2500 )
26 (pin name="clk" layer=2 width=0.0800 depth=0.2500 )
27 (pin name="boot_datai[29]" layer=2 width=0.0800 depth=0.2500 )
28 (pin name="boot_datai[28]" layer=2 width=0.0800 depth=0.2500 )
29 (pin name="boot_datai[27]" layer=2 width=0.0800 depth=0.2500 )
30 (pin name="boot_datai[26]" layer=2 width=0.0800 depth=0.2500 )
31 (pin name="boot_datai[25]" layer=2 width=0.0800 depth=0.2500 )
32 (pin name="boot_datai[24]" layer=2 width=0.0800 depth=0.2500 )
33 (pin name="boot_datai[23]" layer=2 width=0.0800 depth=0.2500 )
34 )
35 (left
36 (pin name="boot_datai[22]" layer=3 width=0.0800 depth=0.2500 )
37 (pin name="boot_datai[21]" layer=3 width=0.0800 depth=0.2500 )
38 (pin name="boot_datai[20]" layer=3 width=0.0800 depth=0.2500 )
39 (pin name="boot_datai[19]" layer=3 width=0.0800 depth=0.2500 )
40 (pin name="boot_datai[18]" layer=3 width=0.0800 depth=0.2500 )
41 (pin name="boot_datai[17]" layer=3 width=0.0800 depth=0.2500 )
42 (pin name="boot_datai[16]" layer=3 width=0.0800 depth=0.2500 )
43 (pin name="boot_datai[15]" layer=3 width=0.0800 depth=0.2500 )
44 (pin name="boot_datai[14]" layer=3 width=0.0800 depth=0.2500 )
45 (pin name="boot_datai[13]" layer=3 width=0.0800 depth=0.2500 )
46 (pin name="rst_n" layer=3 width=0.0800 depth=0.2500 )
47 (pin name="boot_datai[12]" layer=3 width=0.0800 depth=0.2500 )
48 (pin name="boot_datai[11]" layer=3 width=0.0800 depth=0.2500 )
49 (pin name="boot_datai[10]" layer=3 width=0.0800 depth=0.2500 )
50 (pin name="boot_datai[9]" layer=3 width=0.0800 depth=0.2500 )
51 (pin name="boot_datai[8]" layer=3 width=0.0800 depth=0.2500 )
52 (pin name="boot_datai[7]" layer=3 width=0.0800 depth=0.2500 )
53 (pin name="boot_datai[6]" layer=3 width=0.0800 depth=0.2500 )
54 (pin name="boot_datai[5]" layer=3 width=0.0800 depth=0.2500 )
55 (pin name="boot_datai[4]" layer=3 width=0.0800 depth=0.2500 )
56 (pin name="boot_datai[3]" layer=3 width=0.0800 depth=0.2500 )
57 )
)

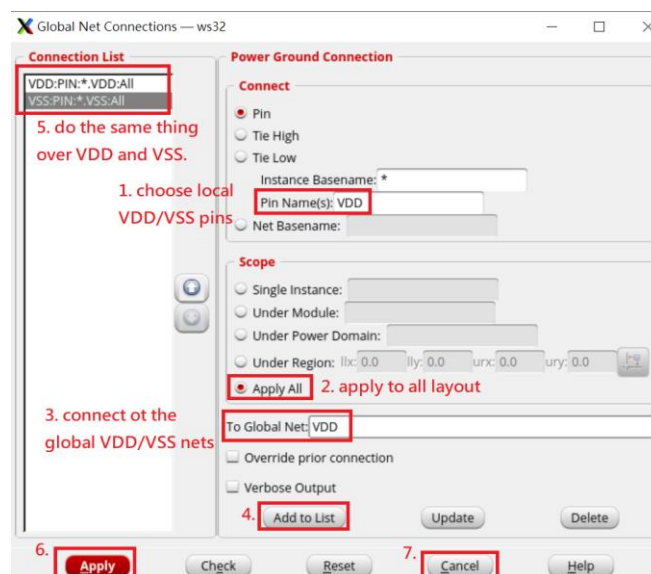
```

5. Set process node

\$innovus > setDesignMode -process 45

6. Power/Ground connection.

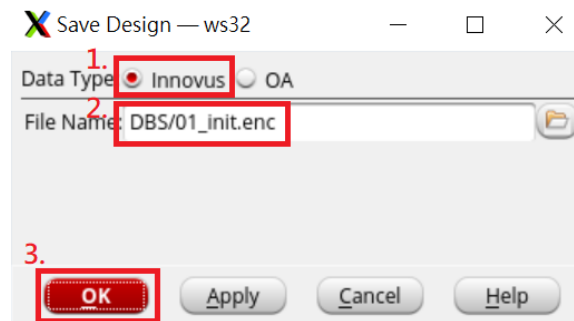
“Power->Connect Global Nets...”



Note: This step connects (logically, not physically routed yet) power and ground for all the nets or pins with names VDD and VSS respectively.

7. Save design.

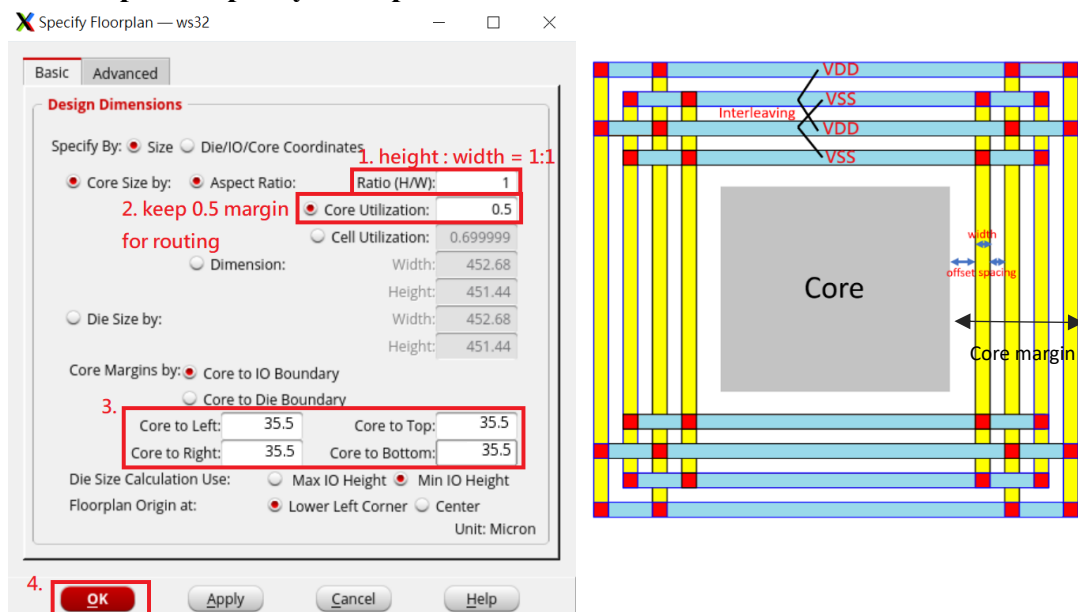
“File->Save Design...”



II. Floorplan

1. Create Floorplan.

“Floorplan->Specify Floorplan...”



Note: “Core to IO boundary” means the space we preserve for core ring. The space should be large enough for offset, space and width of core rings, and should be as small as possible for area concern.

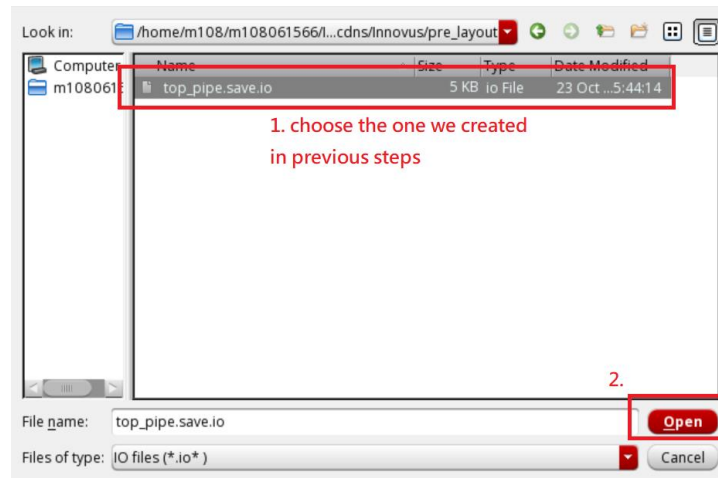
Note: Here we choose 35.5 for

$$\{\text{offset} + 3*2*\text{ring} + 5*\text{spacing} + \text{offset}\} = \{2 + 6*4 + 5*1.5 + 2\} = 35.5$$

Note: a) The core utilization can be set up to 70~80% normally. Here we set 50% only for faster P&R for the lab. b) The core margins are reserved for the core power ring. c) power ring width is set to be 4

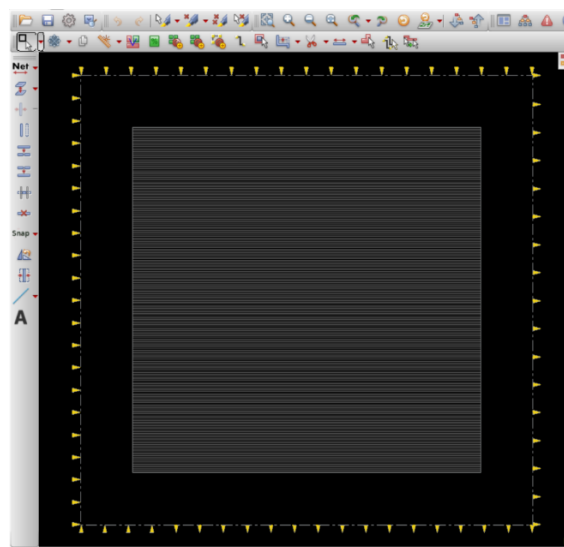
2. Read IO constraint file

“File->Load->I/O File...”

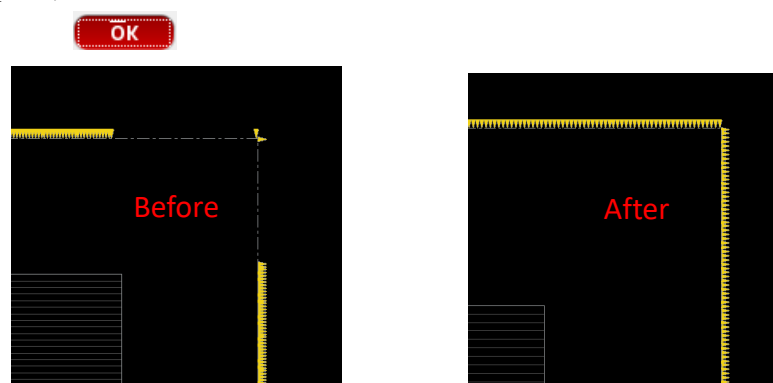


Press ‘f’ or ‘ctrl+r’ to redraw layout

Note: The tool automatically adds IO during floorplan, you still need to reload your IO file again for desired IO pin location.



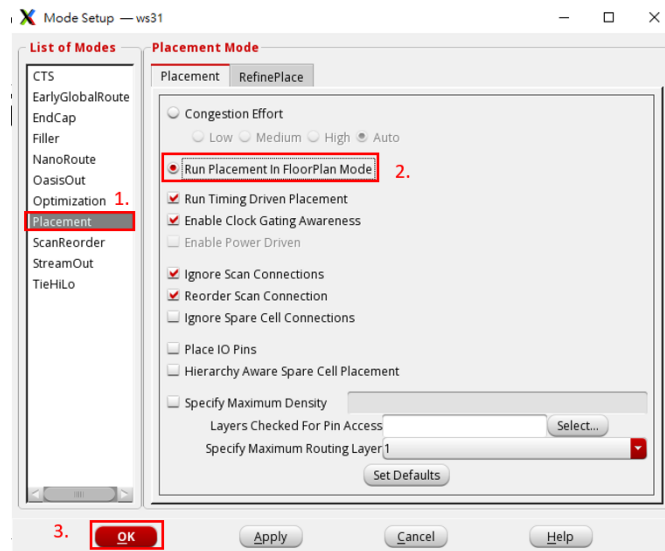
Note: If the IO pins appear to be unbalanced distributed, **Floorplan->Specify Floorplan,**



III. Initial Placement

1. Set placement to floorplan mode

“Tools-> Set Mode->Mode Setup...”



2. Execute initial placement

Type the following command in the terminal

```
$ innovus > place_design
```

```
$ innovus > refinePlace -checkRoute 1
```

3. Early global route

“Route->Early Global Route...”, press 

4. Check timing

“Timing->Report Timing...”

```
timeDesign Summary
-----
Setup views included:
AV_max

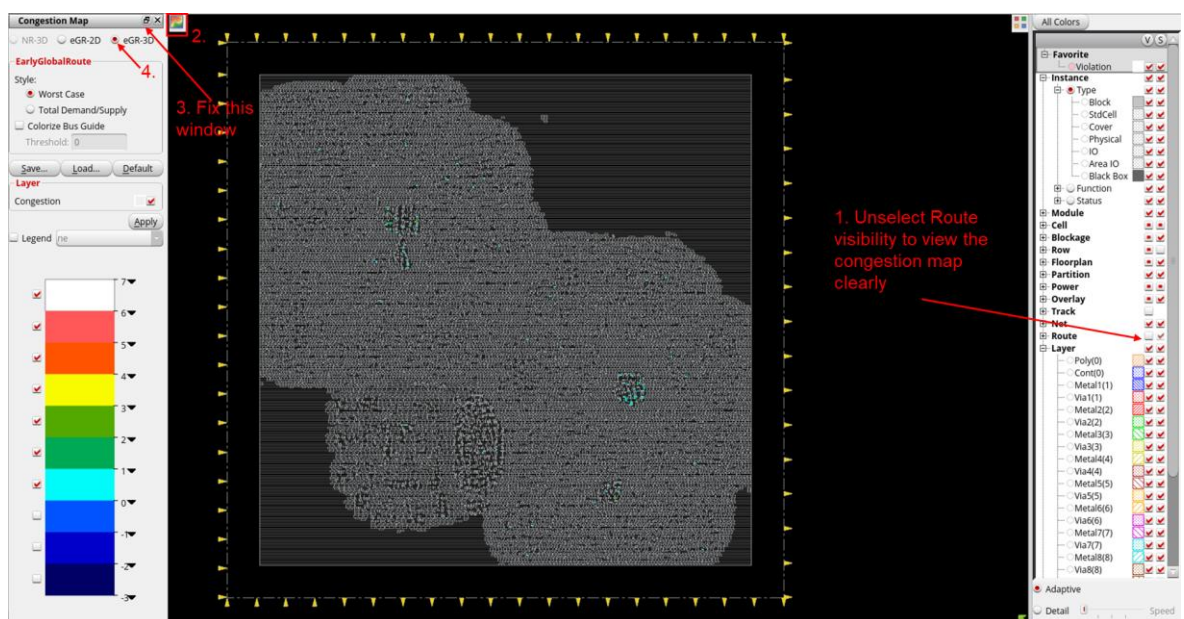
+-----+-----+-----+-----+-----+
| Setup mode | all | reg2reg | reg2cgate | default |
+-----+-----+-----+-----+-----+
| WNS (ns):  | 0.006 | 0.006 | 1.872 | 1.945 |
| TNS (ns):  | 0.000 | 0.000 | 0.000 | 0.000 |
| Violating Paths: | 0 | 0 | 0 | 0 |
| All Paths: | 23435 | 21171 | 2188 | 76 |
+-----+-----+-----+-----+-----+
Density: 48.900%
```


Note: Placement in floorplan mode uses low effort for congestion and applies simplest routing for delay calculation, therefore there should not be any violations. If violations occur, you can:

- Reorder your IO
- Synthesize with a more proper design constraint
- Increase the clock period

5. Analyze congestion

“Route->NanoRoute->Analyze Congestion”



Note: The value of overflow

= #of nets crossing the gcell (global routing cell) - available routing tracks

Does the Congestion map show that our design is congested?

If it is, you can use the following command

```
$ innovus > congRepair
```

Select Route visibility after viewing the congestion map

6. Unplace standard cells and delete routing results **(Please do step 6 & 7 after demo)**

```
$ innovus > unplaceAllInsts
```

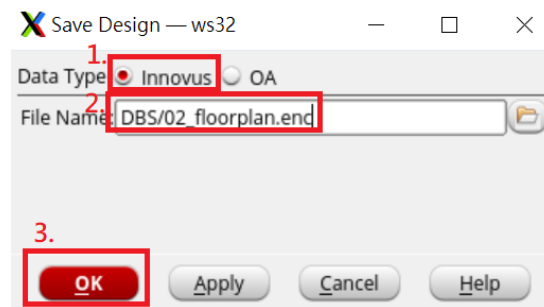


```
$ innovus > editDelete -type Signal
```

Note: We will clean layout after observing congestion map. Formal placement and routing will be performed at later stages.

7. Save your design

“File->Save Design...”



Appendix

Here is the file list of the lab package. Please check if anything missing!

Directory	Filename	Description
innovus/pre_layout/	top_pipe_syn.v	Synthesized netlist
	top_pipe_syn.sdc	Modified SDC for Innovus
	ccopt.cmd	CTS commands
	mmmc.view	MMMC setting file
sim	instruction.txt	CPU simulation pattern
	run_presim.sh	Shell script for pre-simulation
	presim.f	File list for pre-simulation
	run_gatesim.sh	Shell script for gate-level simulation
	gatesim.f	File list for gate-level simulation
	test_top.v	Testbench for the CPU
hdl	top.v	Top module for this CPU
	top_pipe.v	Pipelined design for this CPU
	ID_stage.v	RTL for ID stage
	IF_stage.v	RTL for IF stage
	controller.v	RTL for control signal
	regfile.v	RTL for register file
	ID_EXE.v	RTL for ID to EXE flip flop

	EXE_stage.v	RTL for EXE stage
	alu.v	RTL for alu module
	CPU_define.v	Definition file
	dsram.v	RTL for memory
	PC.v	RTL for program counter
	IF_ID.v	RTL for IF to ID flip flop
syn	.synopsys_dc.setup	DC setup file
	*.tcl	Synthesis scripts
	*.log	Log files
	run_dc.sh	Shell script for synthesis
syn/report	*.out	Report files
	top_pipe_syn.ddc	Netlist & constraints can be read by DC
	top_pipe_syn.saif	Switching activity interchange format (for power)
	top_pipe_syn.sdc	Synopsys design constraints format
	top_pipe_syn.sdf	Standard delay format
	top_pipe_syn.v	Netlist in Verilog
primitime/pre_layout	pt_px.tcl	Script for pre-layout power analysis
primitime/postsim_waveform	pt_px.tcl	Script for post-layout power analysis with post-sim waveform
primitime/presim_waveform	pt_px.tcl	Script for post-layout power analysis with pre-sim waveform
lec/pre_layout_lec	run_lec.sh	Shell script for LEC
	*.do	Scripts for RTL-to-netlist LEC
lec/post_layout_lec	run_lec.sh	Shell script for LEC
	*.do	Scripts for netlist-to-post-layout LEC